

ANALISIS DE ENTRENAMIENTO DEL MODELO GMM

El modelo Mixto Gaussiano es un modelo probabilístico que asume que los datos han sido generados por una mezcla de distribuciones gaussianas.

Las distribuciones gaussianas son campanas de Gauss. Estas campanas son los clusters del conjunto de datos (pulsos Light+Dark). El modelo representa cómo se distribuyen estos pulsos alrededor de la gaussiana (cluster) en función de un conjunto de características.

A priori podríamos pensar que solo habrán dos clústers, uno para los pulsos light y otro para los dark. Pero a lo largo del proyecto se ha visto que dentro de los pulsos dark hay subgrupos. Resulta interesante conocer como el modelo distribuye los pulsos en campanas.

No olvidemos que el objetivo del proyecto es poder diferenciar el ruido de los pulsos de luz reales. La dificultad radica en aquellos fotones de cuerpo oscuro cercanos a los 1064 nm. Quizás con este modelo que extrae características específicas de los pulsos podamos encontrar un patron que cumple estos fotones de cuerpo negro que los pulsos reales No cumple.

Extracción de las features de los pulsos

Para el entrenamiento de este modelo se requiere extraer características específicas de los pulsos para que luego se construyan las gaussianas en función a estas características.

Tomaremos como referencia el artículo, **aquí destacan como características importantes del pulso, la energía del pulso, que está fuertemente relacionada con la amplitud pico mínima; τ_{rise} , el tiempo de subida del pulso y τ_{decay} Tiempo de decaimiento de la señal.**

CONJUNTO DE DATOS DE ENTRENAMIENTO

Es lógico pensar que, siendo el objetivo del proyecto, separar pulsos light y los dark (ruido), debemos entrenar el modelo con ambos conjuntos y que el modelo separe o distribuya cada pulso a la campana que más

se ajuste según las características extraídas de cada pulso. Pero tenemos un problema, el modelo ya no es un detector de anomalías. Se convierte en un

modelo de densidad general que aprende “como son todos los pulsos juntos”. Por tanto, en el ciclo de evaluación, cuando le pasemos un pulso Dark, el modelo no podrá separarlo de los que son “normales” ya que en el proceso de entrenamiento también fue entrenado con Darks y pensará que también son normales. Es decir, el algoritmo intentará encontrar las gaussianas que mejor expliquen la distribución conjunta de las características: v_{min} , τ_{rise} , τ_{decay} .

Por tanto, el procedimiento correcto es entrenar un modelo solo con pulsos Light, así el modelo aprende **cómo** es un pulso normal. Y cuando evaluemos con pulsos DARK, los valores de Log-verosimilitud deberían ser menores.

El segundo modelo, lo entrenamos con solo pulsos DARK para **que aprenda cómo son las anomalías**. En la evaluación del modelo con pulsos Light, los valores de log-verosimilitud deben ser menores.

El tercer modelo, lo entrenamos con pulsos Light+Dark, para que el modelo aprenda la distribución global del detector.

Evaluación: el GMM No es un modelo clasificador, por tanto, no hay que comparar el accuracy.

En su lugar, se comparará el ROC-AUC utilizando como score:

```
score = -gmm.score_samples(...)
```

PREPARACIÓN DEL CONJUNTO DE DATOS

- CARACTERÍSTICAS PARA EXTRAER

Para las características a extraer, en el artículo se indica que las características relevantes son la amplitud del pulso V_{min} , τ_{rise} y τ_{decay} . Por tanto, usaremos estas mismas, de esta manera optimizamos recursos.

- SEPARACIÓN DE CONJUNTOS

Los modelos se tienen que evaluar con pulsos que no han visto durante el entrenamiento. Por tanto, los sets que son los pulsos con las características ya separadas, F_{light} y F_{dark} , se separan siguiendo la distribución 80/20. Es decir, se entrena con el 80 % del conjunto y el 20 % restante es para la evaluación.

Para el modelo GMM entrenado con solo pulsos Light, se entrena con el 80% de F_{light} y se evalúa con el 20% de F_{light} y con el 20 % de F_{dark} .

El modelo GMM entrenado con solo pulsos Dark se aplica el mismo método.

El GMM se entrena con F_both que contiene una mezcla de tipo Light y Dark.

ENTRENAMIENTO DE LAS VARIANTES DE MODELO GMM SEGÚN EL TIPO DE DATO DE ENTRENAMIENTO

GMM entrenado con Light:

F_light_norm_l, F_dark_norm_l, scaler_l = normalizar_caracteristicas(F_light, F_dark)

```
def normalizar_caracteristicas(F_train, F_test=None):
    """
    OBJ: Normalizar las características utilizando únicamente
    las estadísticas del conjunto de entrenamiento.
    PARAMS:
        F_train : (np.ndarray) Características de entrenamiento.
        F_test : (np.ndarray) opcional Características de test.
    RETURNS:
        Si F_test es None: F_train_norm, scaler
        Si F_test no es None: F_train_norm, F_test_norm, scaler
    """

    scaler = StandardScaler()
    F_train_norm = scaler.fit_transform(F_train)
    if F_test is None:
        return F_train_norm, scaler
    F_test_norm = scaler.transform(F_test)

    return F_train_norm, F_test_norm, scaler
```

Crea un objeto de la clase StandardScaler. La función fit_transform() sobre light: calcula media/std de light (F_train) y las aplica.

transform sobre dark (F_test): aplica LA MISMA media/std que calculó con light.

gmm_light = _entrenar_gmm_variante(F_light_norm_l, "Solo Light")

¿Qué es el scaler?

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
F_train_norm = scaler.fit_transform(F_train)
```

El objeto scaler guarda internamente:

- media de cada característica
- desviación típica de cada característica

¿por qué usamos el signo menos en gmm.score_samples()?

Porque queremos construir una ROC. Y en detección de anomalías suele ser más cómodo que:

Score alto -> anomalía

Score bajo -> normal

Sin embargo el GMM devuelve:

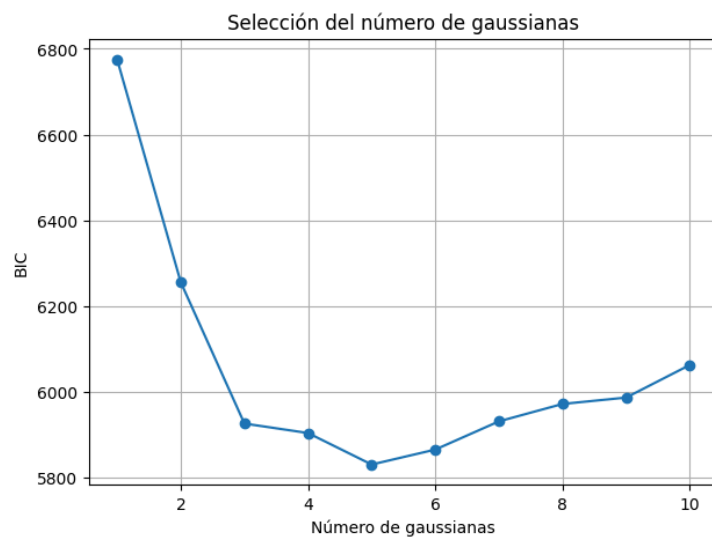
Score alto (menos negativo) -> normal

Score bajo (muy negativo) -> anomalía

ARQUITECTURA DE LOS MODELOS

En el contexto de los Modelos de Mezcla Gaussiana (GMM), el BIC se usa comúnmente para determinar el número óptimo de componentes gaussianos (el valor de `n_components`) a utilizar en el modelo. Se busca el número de componentes que minimicen el BIC. Como cada modelo se entrena con un conjunto diferente de datos, el cálculo del BIC se hace con el conjunto de entrenamiento para cada variante del modelo GMM.

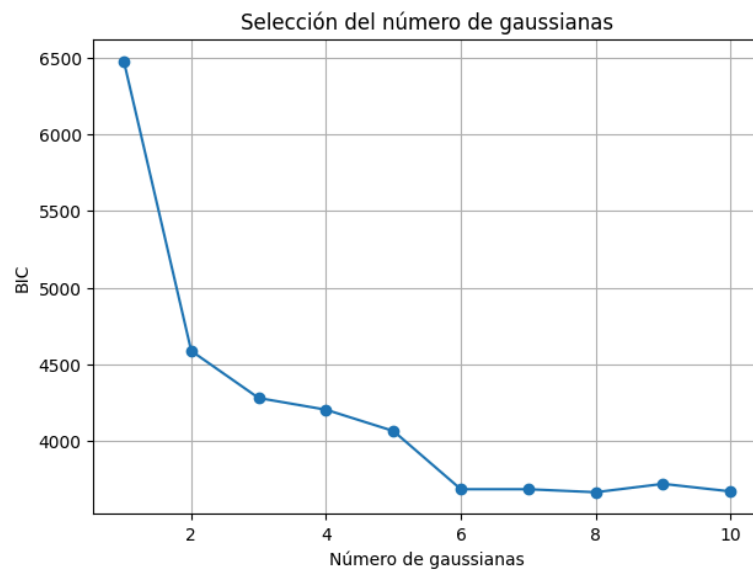
BIC de F_light_train_norm



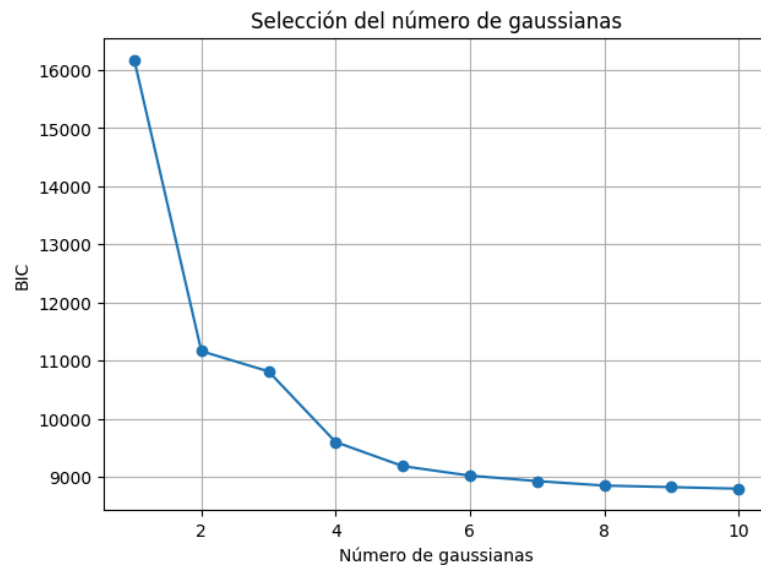
Mejor número de gaussianas: 5

BIC mínimo: 5830.01

BIC de F_train_dark_norm



BIC de F_both_train_norm



EVALUACIÓN DE LOS MODELOS

`score_samples(X)`

Compute the log-likelihood of each sample

La verosimilitud es una medida que indica la probabilidad de que el pulso medido pertenezca a la gaussiana. Como estos valores son muy pequeños, se aplica el logaritmo. **Log-verosimilitud.**

Por cada modelo:

GMM-light

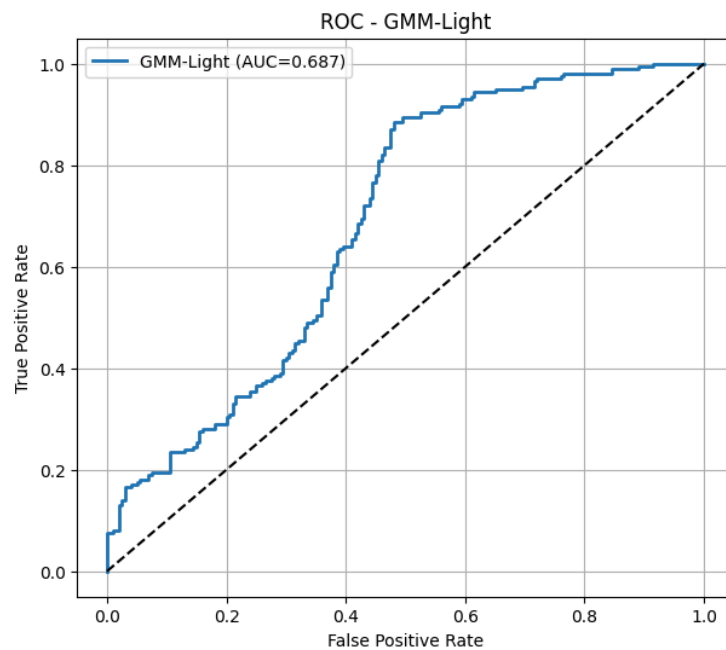
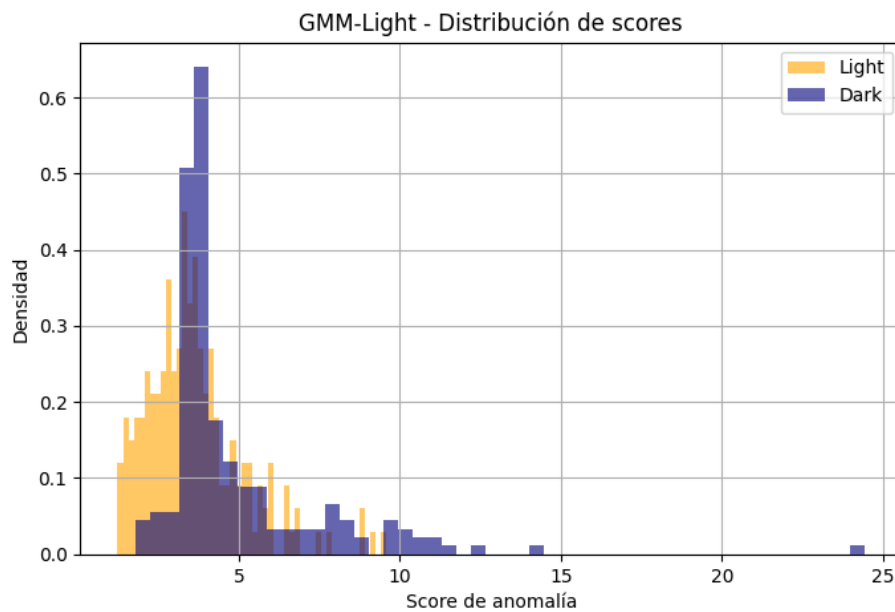
Se guardan los valores de log-verosimilitud en ***scores_light*** y ***scores_dark***.

```
plt.hist(scores_light, bins=50,density=True,alpha=0.6,label="Light")
```

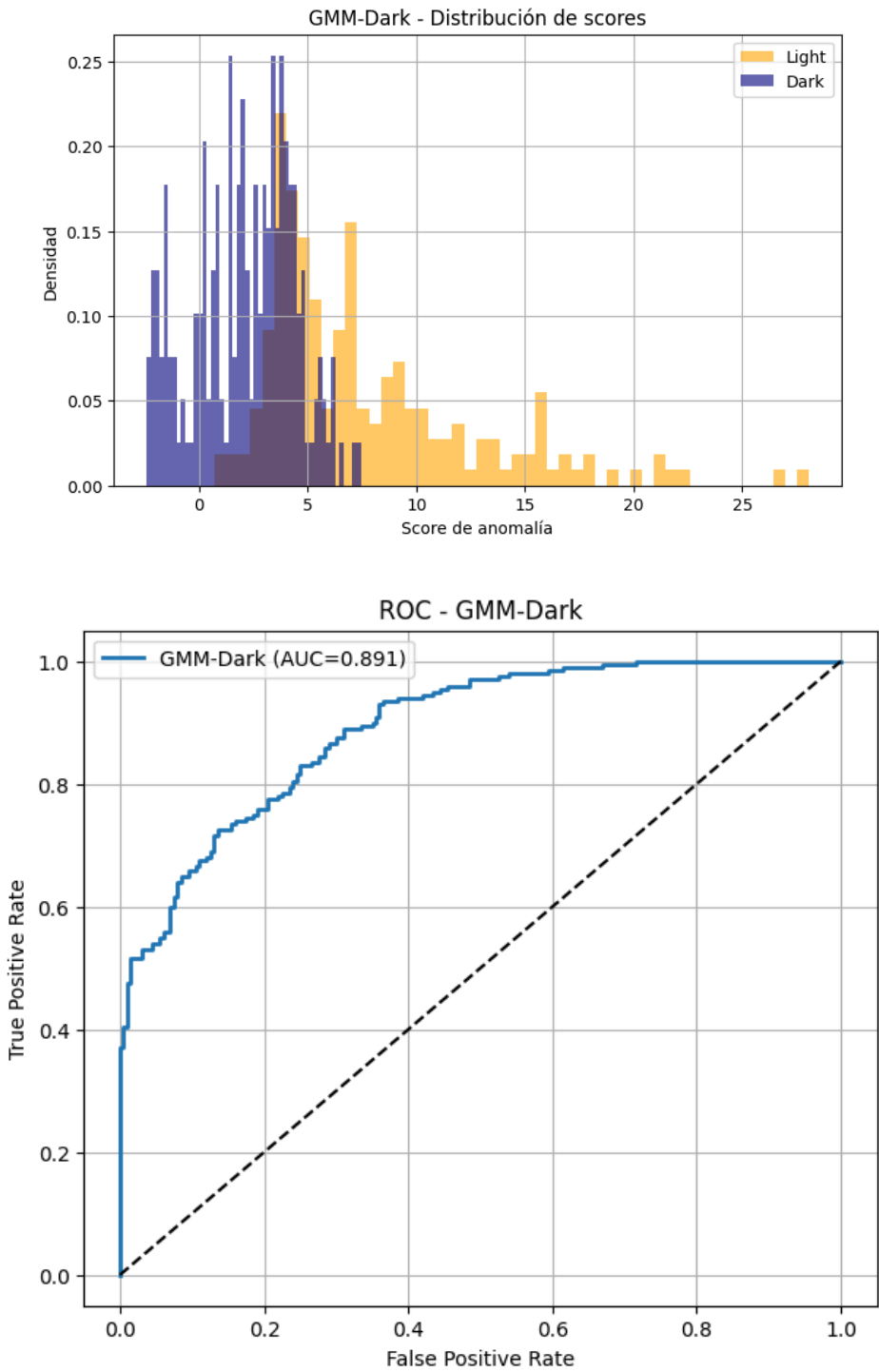
```
plt.hist(scores_dark,bins=50,density=True,alpha=0.6,label="Dark")
```

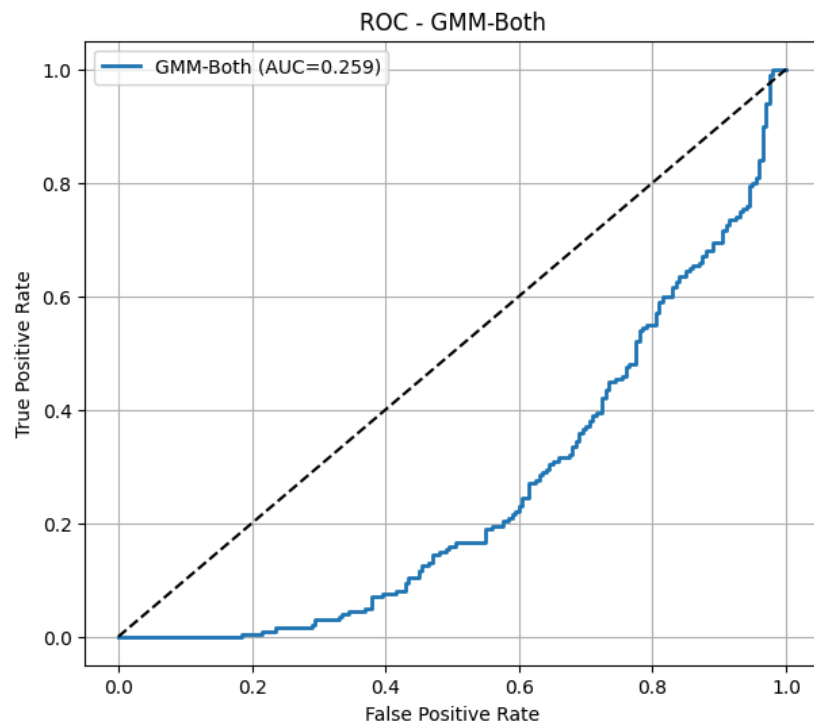
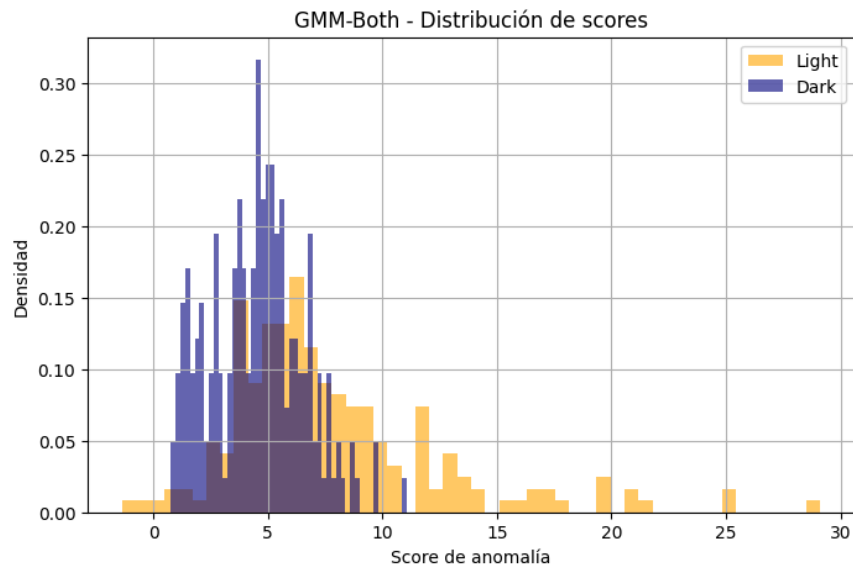
Luego en un histograma se pintan los valores de log-verosimilitud.

HISTOGRAMAS CON UNA GAUSSIANA:



GMM DARK:





Análisis:

1. GMM-Light (entrenado con pulsos Light):

Distribución de Scores: Observando el histograma "GMM-Light - Distribución de scores", vemos que los scores para los pulsos 'Light' (naranja) se concentran en valores más bajos, mientras que los scores para los pulsos 'Dark' (azul) tienen una distribución que se extiende hacia valores más altos. Esto es lo esperado, ya que el modelo fue entrenado para 'Light', por lo que los pulsos 'Dark' deberían considerarse más "anómalos" (scores más altos).

Curva ROC y AUC: La curva ROC "ROC - GMM-Light" muestra un AUC de 0.687. Esto indica que el modelo tiene una capacidad moderada para distinguir entre pulsos 'Light' (considerados "normales") y pulsos 'Dark' (considerados "anomalías") cuando ha sido entrenado solo con datos 'Light'. No es un clasificador perfecto, pero supera una clasificación aleatoria (AUC=0.5).

2. GMM-Dark (entrenado con pulsos Dark):

Distribución de Scores: En el histograma "GMM-Dark - Distribución de scores", los scores de los pulsos 'Dark' (azul) se agrupan en valores más bajos, y los pulsos 'Light' (naranja) tienden a tener scores más altos. De nuevo, esto es consistente, ya que el modelo aprendió la distribución de 'Dark', por lo que 'Light' debería ser más anómalo.

Curva ROC y AUC: La curva ROC "ROC - GMM-Dark" presenta un AUC de 0.891. Este es un resultado significativamente mejor que el GMM-Light. Implica que el GMM-Dark es mucho más efectivo para identificar pulsos 'Light' como anomalías cuando ha sido entrenado exclusivamente con datos 'Dark'.

3. GMM-Both (entrenado con pulsos Light y Dark mezclados):

Distribución de Scores: El histograma "GMM-Both - Distribución de scores" muestra que ambos tipos de pulsos ('Light' y 'Dark') tienen distribuciones de scores bastante mezcladas y superpuestas. Los scores para 'Dark' son un poco más altos, pero la separación no es tan clara como en los modelos entrenados individualmente.

Curva ROC y AUC: La curva ROC "ROC - GMM-Both" arroja un AUC de 0.259. Este es un resultado muy bajo, incluso peor que una clasificación aleatoria. Un AUC por debajo de 0.5 indica que el modelo clasifica peor que el azar, lo que sugiere que para la tarea de distinguir Light vs Dark, este modelo es ineficaz. Probablemente, esto se debe a que al entrenar el GMM con ambos tipos de pulsos, el modelo intenta ajustarse a la distribución general, dificultando la identificación de uno como "anomalía" respecto al otro.

Comparación Final de Modelos GMM (Curva ROC Combinada):

La gráfica "Comparación de modelos GMM" resume el rendimiento:

El GMM-Dark (línea naranja, AUC=0.891) es claramente el modelo con mejor rendimiento. Es el que mejor distingue entre los dos tipos de pulsos, considerando a 'Light' como anomalía.

El GMM-Light (línea azul, AUC=0.687) tiene un rendimiento aceptable, pero inferior al GMM-Dark, identificando 'Dark' como anomalía.

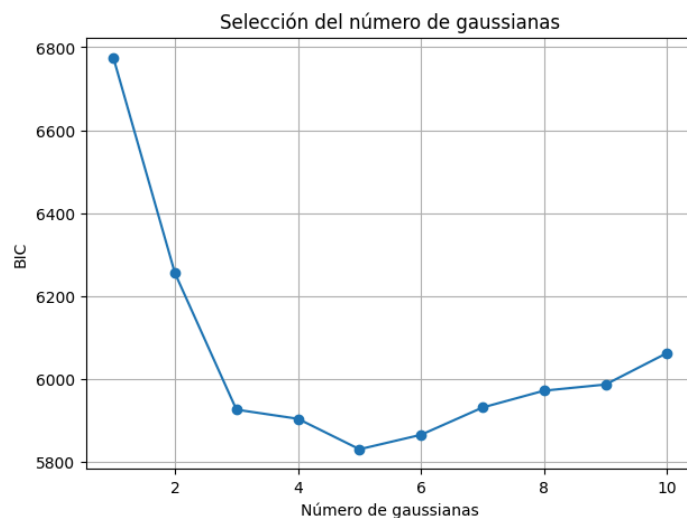
El GMM-Both (línea verde, AUC=0.259) tiene el peor rendimiento, indicando que entrenar el GMM con una mezcla de ambos tipos de datos (y luego tratar de usarlo para discriminar entre ellos como si uno fuera normal y el otro anómalo) no es una estrategia efectiva en este caso. El modelo no logra aprender una representación que separe adecuadamente las clases para la detección de anomalías.

MEDIR EL BIC CON DIFERENTES GAUSSIANAS

. A continuación, calcularemos el BIC con cada conjunto de pulsos (ya con las características extraídas) y nos quedaremos con el menor BIC, es decir, el N° de gaussianas optimo.

Los resultados obtenidos han sido los siguientes:

Para F_light_train_norm

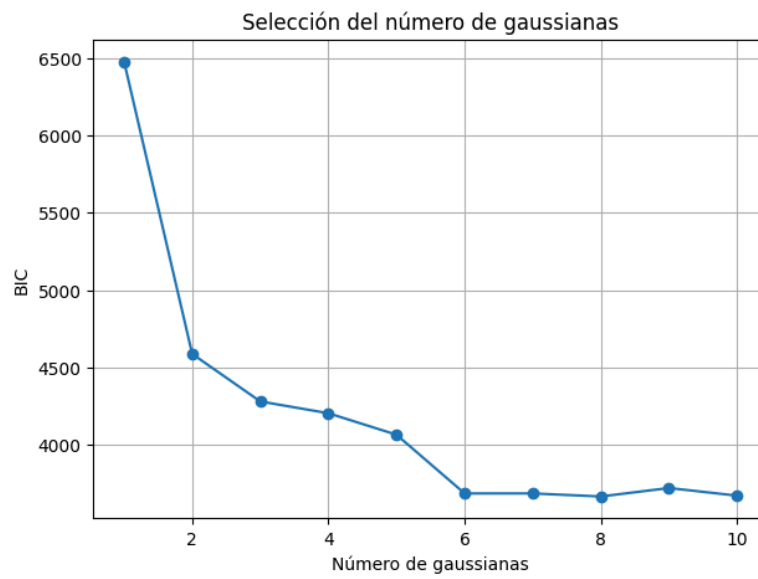


Mejor número de gaussianas: 5

BIC mínimo: 5830.01

```
[np.float64(6774.939072883263),  
 np.float64(6255.26417934118),  
 np.float64(5925.551825493764),  
 np.float64(5903.500529860064),  
 np.float64(5830.011826218114),  
 np.float64(5864.893568582936),  
 np.float64(5930.704315911708),  
 np.float64(5971.3448679449175),  
 np.float64(5986.250010684125),  
 np.float64(6062.261990364279)]
```

Para F_dark_train_norm.

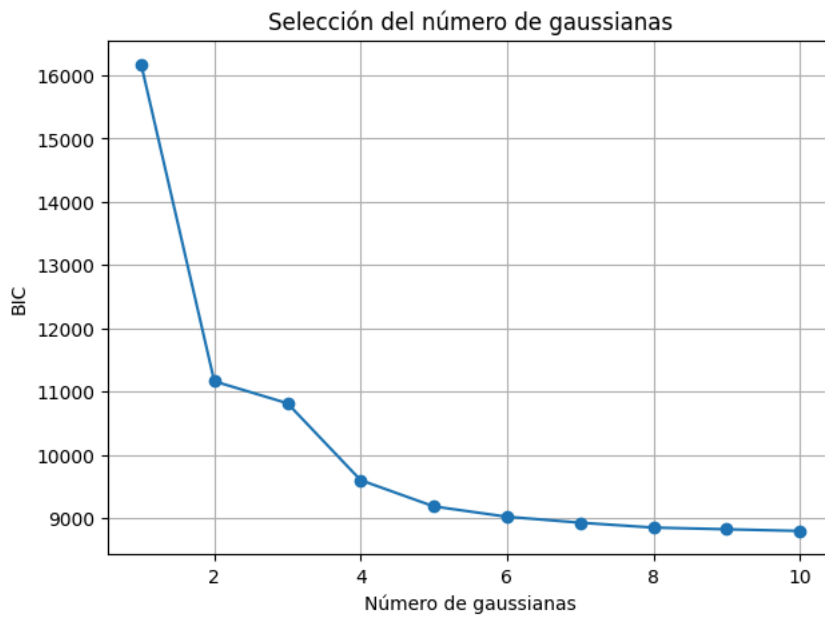


Mejor número de gaussianas: 8

BIC mínimo: 3668.59

```
[np.float64(6474.428523694778),  
 np.float64(4588.607248719832),  
 np.float64(4281.912560974424),  
 np.float64(4205.730681222337),  
 np.float64(4067.0161473694475),  
 np.float64(3688.130658998965),  
 np.float64(3688.0923186992427),  
 np.float64(3668.58633161025),  
 np.float64(3723.1324273375026),  
 np.float64(3673.8764798331363)]
```

Para F_both_train_norm



“El BIC mide de la calidad del ajuste del modelo a los datos con los que aprendió, penalizando la complejidad”

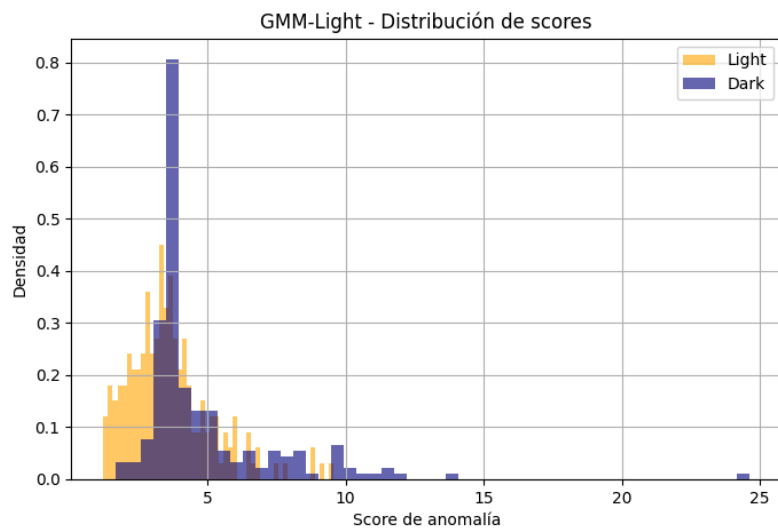
Resultados obtenidos:

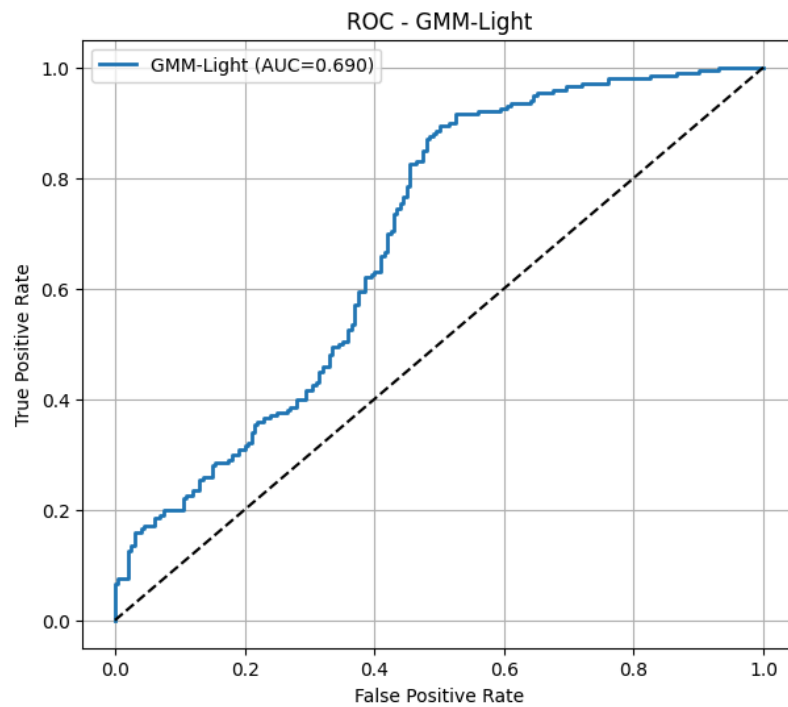
Se ha vuelto a evaluar los modelos esta vez con los nuevos valores de gaussianas.

GMM LIGHT: 5 gaussianas

GMM DARK: 8 gaussianas

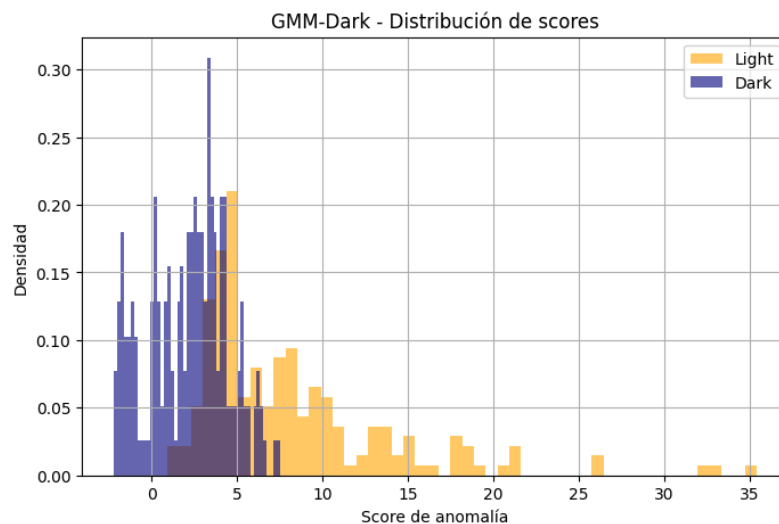
GMM BOTH: 10 gaussianas

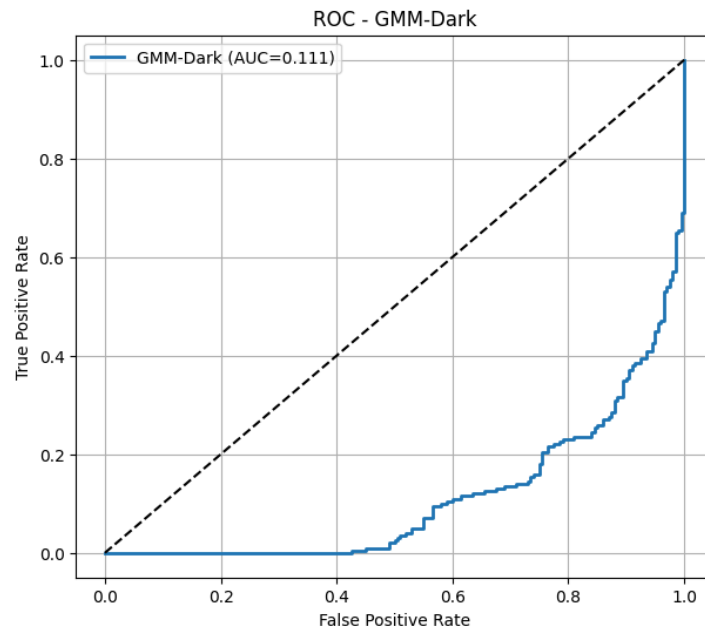




Observamos que para el GMM Light, la proporción de anomalías (DARK) correctamente detectadas es mayor que la de falsos positivos, es decir un pulso LIGHT erróneamente detectado como anomalía. Esto se refleja en el Area Bajo la Curva (AUC) que supera el umbral de un modelo aleatorio (AUC = 0.5).

Sin embargo, vemos que hay mucho solapamiento en los valores de versosimilitud de los pulsos Light y Dark.





Por otro lado, en el modelo GMM DARK, seguimos teniendo mucho solapamiento en los valores de verosimilitud para los pulsos LIGHT y DARK. Además, el valor del área bajo la curva es muy malo, está por debajo al de un modelo aleatorio (AUC = 0.5).

¿Podría ser porque los labels estén mal asignados?

```
# ROC
# =====
scores = np.concatenate([
    scores_light,
    scores_dark
])
y_true = np.concatenate([
    np.zeros(len(scores_light)),
    np.ones(len(scores_dark))
])
fpr, tpr, _ = roc_curve(
    y_true,
    scores
)
roc_auc = auc(fpr, tpr)
```

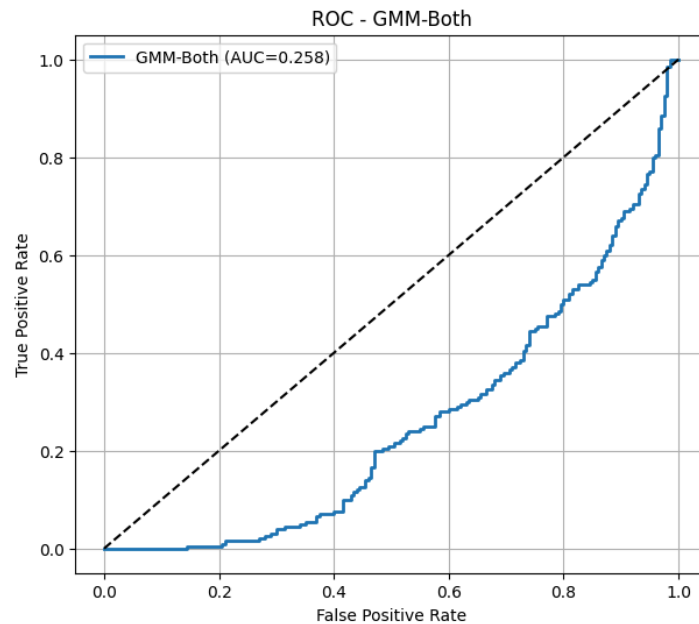
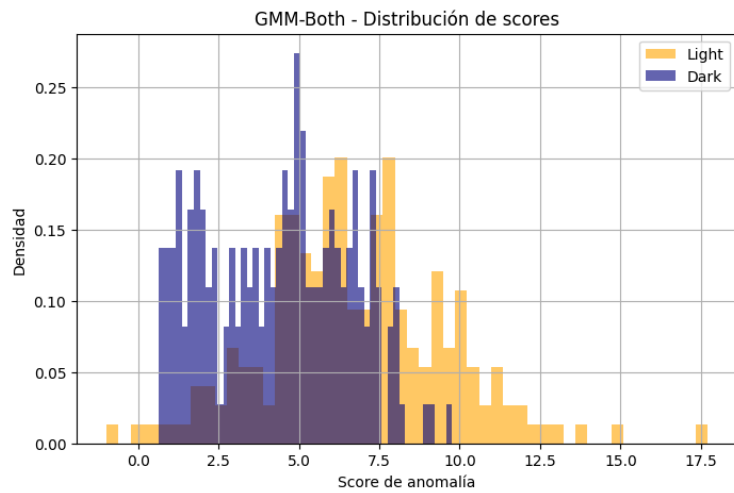
Vemos que se crea un array con dos arrays dentro. Uno con ceros y otro con unos. La longitud de estos subarrays será la misma que `scores_light` y `scores_dark`.

Supongamos que los valores de `scores_light` = [2.1, 1.8, 2.5, 3.0]

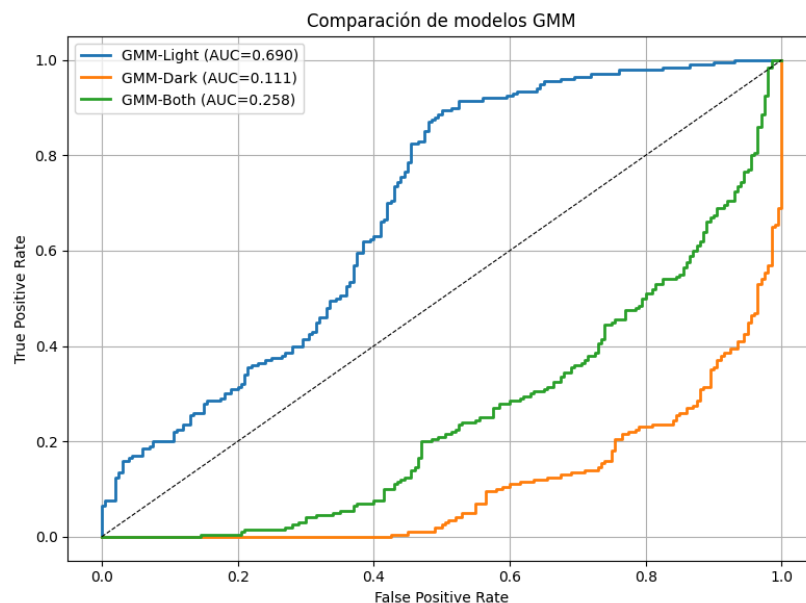
Y para `scores_dark` = [5.8, 4.9, 6.1]

score_samples(x)
Compute the log-likelihood of each sample.

Siguiendo la lógica del GMM entrenado con pulsos Light, los scores más altos (log-verosimilitud alta) tiene una baja probabilidad de pertenecer a la gaussiana, es decir no es del grupo de la gaussiana de Light, por tanto, se considera anomalía.



Finalmente, para el GMM both, tenemos mucho solapamiento de verosimilitudes para los pulos Light y Dark. La curva ROC nos indica que tenemos un rendimiento incluso peor que un modelo aleatorio. Podemos deducir que hay algo que no está yendo bien en el entrenamiento.



Podríamos quedarnos directamente con el modelo GMM Light. Pero que los otros modelos tengan un comportamiento tan malo nos llama especialmente la atención.

Por tanto, vamos a visualizar cómo se distribuyen los pulsos con respecto a las gaussianas, quizás de aquí podamos sacar una conclusión que nos ayude a mejorar los modelos o si finalmente el modelo Mixto Gaussiano No es el indicado para este objetivo.

BUG EN LA EVALUACIÓN DE LOS MODELOS DE GMM

Teníamos un error conceptual en la construcción de los modelos. En cada modelo lo que se considera “normal” y “anomalía” varían en función del conjunto de entrenamiento. Es decir, para el modelo gmm light, que se entrena con pulsos Light, la anomalía son los pulsos DARK. El gmm dark, se ha entrenado con pulsos DARK por tanto se considera anomalía son los pulsos LIGHT. El gmm both (entrenado con ambas clases) es un caso especial, este modelo perdería la capacidad discriminativa ya que interpreta que ambos tipos de pulsos son “normales”, es decir, generaliza. Esperamos que este modelo es el que peor se comporte.

El problema estaba aquí, en la función `evaluar_gmm()`.

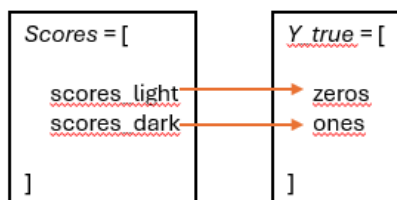
```
# =====
# ROC
# =====

scores = np.concatenate([
    scores_light,
    scores_dark
])

y_true = np.concatenate([
    np.zeros(len(scores_light)),
    np.ones(len(scores_dark))
])

fpr, tpr, _ = roc_curve(
    y_true,
    scores
)
```

Los arrays **scores** e **y_true** tienen la misma longitud. El array **y_true** es el de las 'etiquetas'. La posición de cada etiqueta está relacionada con su score correspondiente:



Esto siempre se construía así. Sin embargo, como se ha comentado antes, en un modelo gmm DARK sabemos que la anomalía es con los pulsos Light. Por tanto, esta asignación ya no nos vale.

Solución:

```
# =====
# ROC
# =====
if nombre_modelo == "GMM-Light":
    scores = np.concatenate([
        scores_light,
        scores_dark # ANOMALIA
    ])
    y_true = np.concatenate([
        np.zeros(len(scores_light)),
        np.ones(len(scores_dark)) # 1-->ANOMALIA
    ])

elif nombre_modelo == "GMM-Dark":
    scores = np.concatenate([
        scores_light, # ANOMALIA
        scores_dark
    ])
    y_true = np.concatenate([
        np.ones(len(scores_light)), # 1-->ANOMALIA
        np.zeros(len(scores_dark))
    ])

else:
    scores = np.concatenate([
        scores_light,
        scores_dark # ANOMALIA
    ])
    y_true = np.concatenate([
        np.zeros(len(scores_light)),
        np.ones(len(scores_dark)) # 1-->ANOMALIA
    ])
```

Condicionales para que la construcción de los arrays dependa del modelo a evaluar.

ANEXO

Método `scaler.fit_transform()`

Combina de forma eficiente dos operaciones secuenciales:

1. **fit()**: Analiza los datos de entrada para aprender las métricas internas. Por ejemplo, si usas `StandardScaler`, calcula la **media (μ)** y la **desviación estándar (σ)** de cada columna.
2. **transform()**: Aplica esos parámetros matemáticos calculados para modificar los valores de los datos. Siguiendo el ejemplo anterior, resta la media y divide por la desviación estándar en cada celda